

Cherthala Water Watch

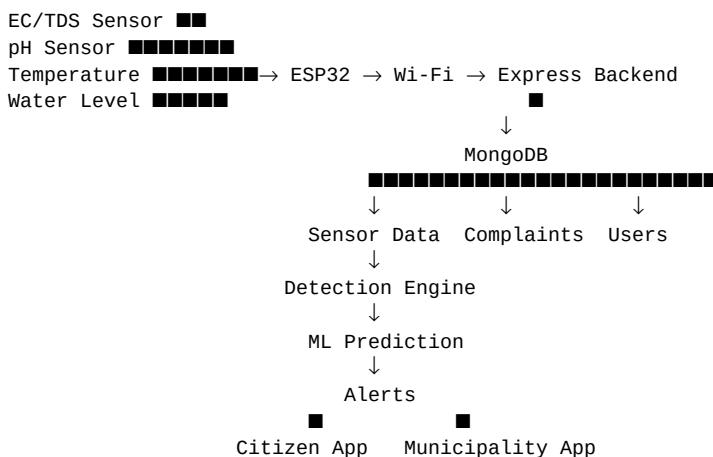
Complete Implementation Roadmap — IoT + Mobile App + Municipality Grievance System + AI/ML

Project objective: Build a smart system that continuously monitors water quality, detects saline water intrusion, predicts intrusion risk, alerts users and authorities, provides live/historical visualization, and allows citizens to register and track complaints.

1. Technology Stack

Mobile App: React Native + Expo
Backend: Node.js + Express
Database: MongoDB
IoT: ESP32 + EC/TDS + pH + temperature + water-level sensors
Communication: ESP32 → REST API → Backend
ML: Python + scikit-learn
Notifications: Firebase Cloud Messaging
Location: GPS / monitoring-station coordinates

2. Complete System Architecture



3. Sensor Parameters

EC/Conductivity: Primary parameter for identifying increased dissolved salts and saline intrusion.

TDS: Supporting indicator of dissolved substances; may be derived from conductivity depending on the sensor and calibration approach.

pH: General water-quality parameter and supporting ML feature; it should not be treated as a standalone saline-intrusion indicator.

Water level: Helps study the relationship between water-level/tidal changes and salinity.

Temperature: Useful for interpreting and compensating sensor measurements and as an ML feature.

4. Implementation Roadmap

Part 1 — Software and IoT simulation

Use Wokwi to simulate an ESP32 and sensor values. First target: ESP32 receives simulated EC/TDS, pH, temperature and water-level values and displays them in the Serial Monitor.

Part 2 — Physical sensor integration

Connect the ESP32 to the actual EC/conductivity, pH, temperature and water-level hardware. Calibrate sensors and validate readings.

Part 3 — Data transmission

Configure ESP32 Wi-Fi and send readings to the Express REST API.

Part 4 — Database

Create MongoDB collections/models for User, SensorReading, MonitoringStation, Complaint, Alert and Prediction.

Part 5 — Citizen mobile app

Implement registration/login, home screen, live monitoring, station details, alerts, complaint registration, complaint history and profile.

Part 6 — Municipality interface

Provide authorized officials with live monitoring, station status, alerts, analytics and complaint management.

Part 7 — Intrusion detection

Implement a rule-based detection layer first: Normal, Warning and Critical. Thresholds must be established from appropriate standards and local calibrated measurements, not arbitrary values.

Part 8 — Historical data collection

Store timestamped sensor readings and environmental information. This dataset becomes the basis for ML.

Part 9 — AI/ML prediction

Clean data, engineer features, train and evaluate models such as Random Forest. Predict intrusion risk or risk class using historical and live measurements.

Part 10 — Integration and testing

Connect live sensor data to the ML service, display predictions in the app, trigger notifications, test failures and validate the complete pipeline.

5. Suggested Database Data

```
SensorReading
{
  timestamp,
  stationId,
  ec,
  tds,
  ph,
```

```

    temperature,
    waterLevel
}

Complaint
{
  userId,
  category,
  description,
  photo,
  location,
  status,
  createdAt,
  updatedAt
}

Prediction
{
  stationId,
  timestamp,
  riskProbability,
  riskLevel,
  predictionHorizon
}

```

6. Citizen App

```

Splash
Login / Register
Home
  ■■■■ Current Water Quality
  ■■■■ Nearby Monitoring Stations
  ■■■■ Current Risk
  ■■■■ AI Prediction
  ■■■■ Latest Alerts
Live Monitoring
Station Details
Alerts
Register Complaint
My Complaints
Profile

```

7. Municipality App / Admin Interface

```

Admin Login
Dashboard
  ■■■■ Monitoring Stations
  ■■■■ Live Sensor Data
  ■■■■ Intrusion Alerts
  ■■■■ AI Predictions
  ■■■■ Complaints
  ■■■■ Complaint Details
  ■■■■ Assign / Investigate
  ■■■■ Update Status

Complaint status:
Submitted → Under Investigation → Action Taken → Resolved

```

8. IoT Data Flow

```

Sensor values
↓
ESP32
↓
Wi-Fi
↓
POST /api/sensor/readings
↓
Express Backend
↓
MongoDB
↓

```

Detection + ML
↓
Mobile App + Notifications

9. ML Pipeline

Historical sensor/environmental data
↓
Data cleaning
↓
Feature engineering
↓
Train / test split
↓
ML model training
↓
Model evaluation
↓
Live prediction API
↓
Risk level / probability
↓
Mobile App

Potential features include EC, TDS, pH, temperature, water level, previous EC/TDS readings, rate of EC change, time/date, rainfall and tidal/water-level information when reliable data is available.

10. Recommended Development Strategy

Do not begin with the ML model or physical hardware. First prove the software pipeline using simulated sensor data. Then replace simulated readings with physical sensors. At the same time, continuously collect historical data. Once sufficient, reliable data exists, train and validate the ML model.

Phase 1: Wokwi simulation
↓
Phase 2: Backend + MongoDB
↓
Phase 3: Mobile app
↓
Phase 4: Physical sensors
↓
Phase 5: Alerts + grievance system
↓
Phase 6: Historical dataset
↓
Phase 7: ML prediction
↓
Phase 8: Final integrated prototype

11. First Milestone

The first implementation target is a Wokwi ESP32 simulation that generates sensor readings for EC/TDS, pH, temperature and water level. The next milestone is to send those readings to a Node.js/Express backend and save them in MongoDB. This establishes the foundation for the mobile app and future ML system.